Here is an end-to-end prompt engineered for Antigravity (or an advanced coding agent) to build the complete project deliverables matching the MAC F244 course guidelines:

```
Markdown
# TASK: Complete Implementation of MAC F244 Course Project (Topic 2: Mean-Reverting Stochastic Processes: OU and CIR Models)

Act as an expert quantitative researcher and financial engineer. Implement the complete project titled:
"Development and Evaluation of Algorithmic Trading Strategies using Mean-Reverting Stochastic Processes: Ornstein-Uhlenbeck and Cox-Ingersoll-Ross Models."

Meet every criterion specified in the course evaluation guidelines:
1. Mathematical formulation and theoretical explanation.
2. Production-grade, well-commented Python code executed inside a virtual environment.
3. Realistic numerical simulation, parameter estimation (MLE/OLS), real financial market data fetching, strategy backtesting, and performance attribution.
4. A concise, publication-grade 3–4 page LaTeX project report (`report.tex` compiled to `report.pdf`) and an accompanying project proposal markdown.
5. Presentation/Viva preparation notes covering likely questions on proofs, discretization, and empirical trade-offs.

---

### Step 1: Environment Setup & Directory Architecture
Initialize a dedicated Python virtual environment and generate the following project structure:

```bash
python3 -m venv venv
source venv/bin/activate
pip install numpy scipy pandas matplotlib seaborn yfinance statsmodels cvxpy tabulate
```

Create this file layout:

Plaintext
├── proposal.md
```

```
├── requirements.txt
├── venv/
├── src/
│   ├── __init__.py
│   ├── models.py          # SDE simulation: OU and CIR via Euler-Maruyama & exact transitions
│   ├── calibration.py     # Maximum Likelihood Estimation (MLE) & Regression calibration
│   ├── strategy.py        # Mean-reversion trading rules, dynamic thresholding, stop-loss
│   ├── backtest.py        # Performance engine: Sharpe, Sortino, Max Drawdown, CAGR
│   └── data_loader.py     # yfinance ingestion (e.g., pairs/spreads, yield proxies, or FX)
├── run_experiments.py     # Master driver running full simulation, empirical tests, and plot generation
├── figures/               # Output plots (trajectories, calibration fits, equity curves, drawdown)
├── report/
│   ├── report.tex         # 3 to 4 page rigorous LaTeX report
│   └── report.pdf
└── viva_preparation.md    # Comprehensive viva Q&A guide (15-20 min presentation prep)
```

Step 2: Mathematical Rigor & Algorithms (src/)

1. **Stochastic Formulations**:
   Ornstein-Uhlenbeck (OU):

$$dX_t = \theta(\mu - X_t)dt + \sigma dW_t$$

   - Detail analytical solutions, transition densities $X_{t+\Delta t} \mid X_t \sim \mathcal{N}\left(\mu + (X_t - \mu)e^{-\theta \Delta t}, \frac{\sigma^2}{2\theta}(1 - e^{-2\theta \Delta t})\right)$, and Euler-Maruyama discretization.
   Cox-Ingersoll-Ross (CIR):

$$dr_t = \kappa(\theta - r_t)dt + \xi\sqrt{r_t}dW_t$$

   - Specify the Feller condition ($2\kappa\theta > \xi^2$) to guarantee positivity, non-central chi-squared exact transition dynamics, and boundary-handling numerical schemes (e.g., absorption/reflection or truncated Euler).
2. **Calibration Engine (calibration.py)**:
   - Implement analytical Maximum Likelihood Estimation (MLE) for both OU and CIR processes from discrete observations.
   - Include Ordinary Least Squares (OLS) AR(1) representation for OU validation.
3. **Trading Strategies (strategy.py)**:

- Construct trading logic on mean-reverting spreads/assets (e.g., standardized Z-scores $Z_t = \frac{X_t - \mu}{\sigma_{eq}}$):
  - Entry rules: Long when $Z_t \leq -z_{entry}$, Short when $Z_t \geq z_{entry}$.
  - Exit rules: Cross back toward mean ($|Z_t| \leq z_{exit}$) or stop-loss trigger ($|Z_t| \geq z_{stop}$).
- CIR Strategy: Volatility-adjusted interest rate / yield ETF regime-switching or mean-reverting swing strategy.

4. **Empirical Validation (run_experiments.py)**:
   - Download real financial data (e.g., cointegrated pair spread like EWA/EWC or interest rate proxies like SHY/IEF).
   - Run sensitivity analysis across varying parameters (θ, σ, transaction costs/slippage).
   - Export high-resolution publication-quality PNG figures into figures/.

Step 3: Formal Project Report (report/report.tex)

Write a clean, dense 3 to 4-page LaTeX document adhering to standard IEEE or SIAM style containing:

- **Title, Authors, Abstract**: Clear focus on OU/CIR calibration and empirical backtesting.
- **Section 1: Introduction & Literature**: Financial context, continuous-time limits of mean-reversion.
- **Section 2: Mathematical Formulation & SDE Discretization**: Closed-form densities, Feller condition proof/significance, Euler vs. exact simulation.
- **Section 3: Parameter Calibration Methodology**: Explicit log-likelihood derivations for discrete sample paths.
- **Section 4: Trading Strategy & Empirical Results**: Data description, equity curves, annualized returns, Sharpe ratio, max drawdown, and sensitivity tables.
- **Section 5: Conclusion & Risk Limitations**: Hedging errors, regime shifts, and structural breaks.
- **References**: Proper academic citations.

Compile the document to report.pdf.

Step 4: Submission Documents

1. **proposal.md**: Fill out the submission template matching the October 31st requirement:
   - Project title & description
   - Mathematical models investigated
   - Computational methodology
   - Expected outcomes
   - Placeholder group member details (Group of 5)
2. **viva_preparation.md**: Detail key viva questions and answers covering:
   - Derivation of the OU mean and variance using Itô's Lemma.
   - Behavior when the Feller condition is violated in CIR.
   - Contrast between Euler-Maruyama discretization bias and exact sampling.
   - Realistic market frictions (slippage, borrowing fees, execution lag).

Execute the code, generate all charts and files, verify error-free execution, and confirm output files are in place.